PROPUESTA TECNICA DE ARRANQUE

Sistema de gestion, seguimiento y evidencia de rondines de prefectos

FIME - UANL | Version beta / documento base para Codex

Proposito del documento
Servir como contexto inicial para que Codex analice el problema, proponga una arquitectura razonable y permita iniciar un prototipo funcional sin inventar requisitos aun no confirmados. Se distinguen decisiones confirmadas, supuestos de trabajo y preguntas pendientes.

Estado del proyecto: etapa temprana de definicion. El equipo de programacion ya propuso C#/.NET, pero la ingeniera responsable aun no ha podido proporcionar todos los detalles operativos.

Fecha del documento: 8 de octubre de 2026

1. Contexto y problema

Problema inicialmente planteado: la idea original contemplaba verificar la presencia del personal administrativo durante la jornada. Despues de una reunion con el equipo y la ingeniera, el alcance fue reducido y definido exclusivamente alrededor de los prefectos y sus rondines.

Alcance confirmado por el equipo
La aplicacion se centrara unicamente en la gestion de rondines de prefectos y en los reportes/incidencias encontrados durante dichos recorridos. No se considera como objetivo de la beta controlar al resto del personal administrativo ni sustituir el checador institucional.

Necesidad principal: dar evidencia, trazabilidad y seguimiento a los rondines para que la coordinacion pueda saber que prefecto realizo un recorrido, cuando lo hizo, que puntos reviso y que incidencias reporto.

2. Objetivo de la beta

Objetivo funcional: construir una primera version operativa que permita registrar el ciclo completo de un rondin, desde su asignacion/inicio hasta su cierre, y generar incidencias asociadas con su correspondiente seguimiento.

1. Autenticar al usuario y reconocer su rol.
2. Mostrar al prefecto los rondines que tiene asignados o disponibles.
3. Permitir iniciar un rondin y registrar el avance por punto de revision.
4. Permitir registrar una incidencia desde un punto concreto del recorrido.
5. Permitir adjuntar descripcion y dejar preparada la evidencia fotografica para una etapa posterior o para la beta, segun viabilidad.
6. Permitir cerrar el rondin y conservar un historial consultable.

7. Permitir a coordinacion/administracion consultar rondines, incidencias y estados.

3. Que sabemos y que NO debemos asumir

Elemento	Estado	Tratamiento en desarrollo
Enfoque en prefectos y rondines	CONFIRMADO	Es el nucleo del MVP.
Reportes de desperfectos/incidencias	CONFIRMADO	Debe existir desde la beta.
Seguimiento administrativo de incidencias	CONFIRMADO	Debe existir un estado y un historial basico.
Roles/permisos	CONFIRMADO a nivel conceptual	Implementar al menos Prefecto y Administrador/Coordinacion.
Edificios y puntos de revision	ALTAMENTE PROBABLE	Modelarlos de forma configurable; no codificar edificios fijos.
QR para validar ubicacion	PENDIENTE	Dejar arquitectura preparada; no tratarlo como requisito cerrado.
GPS/geolocalizacion continua	PENDIENTE	No implementar en MVP salvo instruccion expresa.
Fotografias en incidencias	PENDIENTE	Preparar soporte, pero validar almacenamiento y privacidad.
Horarios y frecuencia de rondines	PENDIENTE	Modelar asignaciones de forma configurable.
Tipos exactos de incidencias	PENDIENTE	Usar catalogo editable o lista inicial simple.
Integracion con UANL/checador	FUERA DE ALCANCE	No incluir en la beta.

Regla de trabajo para Codex
Cuando un requisito este marcado como PENDIENTE, no debe convertirse en una dependencia rigida del sistema. Implementar interfaces, campos opcionales o puntos de extension donde tenga sentido, y documentar la decision tomada.

4. Actores y permisos iniciales

Rol	Responsabilidad	Funciones MVP
Prefecto	Ejecuta recorridos y reporta hallazgos.	Ver rondines asignados; iniciar; revisar puntos; reportar incidencia; cerrar; consultar historial propio.
Coordinacion / Supervisor	Supervisa la operacion.	Ver rondines por fecha/edificio/prefecto; revisar incidencias; cambiar estado; consultar historial.
Administrador	Configura el sistema.	Gestionar usuarios, roles, edificios, puntos, catalogos y asignaciones; acceso total.

5. Flujo principal propuesto

1. Acceso	2. Asignacion	3. Ejecucion	4. Incidencia	5. Cierre	6. Seguimiento
Login	Rondin asignado	Puntos a revisar	Reportar desde el punto	Resumen y cierre	Coordinacion consulta

Ejemplo: el prefecto inicia un rondin del Edificio A a las 09:00. El sistema muestra 8 puntos de revision. En el punto "Baño Hombres - Piso 2" detecta falta de jabon, registra la incidencia y continua. Al terminar, el sistema guarda hora de inicio, hora de fin, puntos revisados e incidencias.

6. Modulos de la beta

Modulo	Objetivo
Autenticacion	Inicio de sesion, cierre de sesion, proteccion por rol.
Dashboard Prefecto	Rondin actual, proximos rondines, historial resumido e incidencias propias.
Gestion de Rondines	Asignacion, inicio, progreso, cierre y consulta historica.
Puntos de Revision	Edificio, area, orden, tipo de punto y resultado de revision.
Incidencias	Registro, prioridad, descripcion, ubicacion, responsable/autor, estado e historial.
Dashboard Administrativo	Resumen por fecha, edificio y prefecto; rondines pendientes/en curso/completados; incidencias abiertas/cerradas.
Catalogos / Configuracion	Edificios, puntos, tipos de incidencia, prioridades, usuarios y asignaciones.

7. Reglas de negocio preliminares

- Un rondin debe tener un prefecto responsable y una ubicacion/edificio.
- Un rondin no debe poder cerrarse si el sistema detecta puntos obligatorios sin revisar, salvo que exista una accion explicita de justificar/omitir.
- Cada punto revisado debe conservar fecha y hora de registro.
- Una incidencia debe quedar asociada al rondin y, cuando sea posible, al punto exacto donde fue detectada.
- Las incidencias deben tener un estado minimo: Pendiente, En proceso y Resuelta.
- Los cambios relevantes (inicio, cierre, cambio de estado de incidencia) deben conservar trazabilidad.
- Los permisos se deben validar en el servidor, no solo ocultando botones en la interfaz.
- El sistema debe evitar que un prefecto modifique historiales cerrados sin una accion administrativa expresa.
- Los edificios y puntos deben ser datos configurables, no valores quemados en codigo.

8. Modelo de datos inicial sugerido

Entidad	Campos sugeridos	Relacion principal
User	Id, Name, Email/UserName, Active, RoleId	Pertenece a un rol.
Role	Id, Name	Admin, Coordinacion, Prefecto.
Building	Id, Name, Code, Active	Contiene puntos de revision.
Checkpoint	Id, BuildingId, Name, Type, Order, Active, OptionalCode	Pertenece a un edificio.
Rondin	Id, PrefectId, BuildingId, ScheduledAt, StartedAt, FinishedAt, Status, Notes	Tiene muchos CheckpointChecks e Incidents.
CheckpointCheck	Id, RondinId, CheckpointId, CheckedAt, Result, Notes	Registra la revision de un punto.
Incident	Id, RondinId, CheckpointId, TypeId, Priority, Description, CreatedAt, Status,	Puede asociarse a un punto.

	ResolvedAt, ResolutionNotes	
IncidentType	Id, Name, Active	Catalogo configurable.
IncidentHistory	Id, IncidentId, StatusFrom, StatusTo, ChangedBy, ChangedAt, Comment	Trazabilidad de incidencias.
RondinAssignment	Id, PrefectId, BuildingId, ScheduledDate, StartWindow, EndWindow, Active	Asigna rondines sin fijarlos al codigo.

Decisiones abiertas del modelo de datos
Aun falta definir si un rondin siempre pertenece a un solo edificio, si hay rondines con varios edificios, cuantos puntos tiene cada recorrido, y si las asignaciones se generan diariamente, semanalmente o manualmente. El esquema anterior permite empezar sin bloquear esas decisiones.

9. Arquitectura tecnica recomendada para la beta

Stack confirmado por el equipo: C#/.NET. Como base de trabajo actual, se propone .NET 10 LTS con ASP.NET Core y Entity Framework Core 10. Microsoft identifica .NET 10 como version LTS; EF Core 10 fue publicado en noviembre de 2025 y tambien es LTS.

- Backend: ASP.NET Core en .NET 10.
- UI beta recomendada: Blazor Web App responsiva para poder usar la misma base C# en navegador de escritorio y telefono. Esta eleccion es una propuesta, no un requisito validado.
- Acceso a datos: Entity Framework Core 10.
- Base de datos: SQL Server como opcion inicial por afinidad con el ecosistema Microsoft; cambiar si la infraestructura institucional exige otro motor.
- Autenticacion/autorizacion: ASP.NET Core Identity + roles.
- Arquitectura: separacion por capas ligera (Web/UI, Application, Domain, Infrastructure) sin sobreingenieria.
- Pruebas: unitarias para reglas de negocio y pruebas de integracion para persistencia/endpoints criticos.
- Auditoria: conservar timestamps y usuario que realiza acciones relevantes.
- Configuracion: appsettings + variables de entorno/secrets; nunca credenciales incrustadas en el codigo.

Por que esta base
ASP.NET Core ofrece autenticacion, autorizacion, inyeccion de dependencias, configuracion, logging y soporte para aplicaciones web; EF Core aporta un ORM para trabajar con entidades .NET y bases de datos. Esto permite construir una beta con una pila coherente y mantenible sin introducir tecnologias innecesarias.

10. Estructura de solucion sugerida

```
Fime.Rondines.sln
  src/
    Fime.Rondines.Web/          # UI, autenticacion y endpoints
    Fime.Rondines.Application/   # casos de uso, DTOs, validaciones
    Fime.Rondines.Domain/        # entidades y reglas de negocio
    Fime.Rondines.Infrastructure/ # EF Core, SQL Server, repositorios, almacenamiento
  tests/
```

```
  Fime.Rondines.UnitTests/
  Fime.Rondines.IntegrationTests/
docs/
  requirements.md
  decisions.md
```

11. Pantallas base de la beta

Pantalla	Usuario	Contenido minimo
Login	Todos	Usuario, contrasena, acceso.
Inicio Prefecto	Prefecto	Rondin activo/proximo, estado e incidencias recientes.
Detalle de Rondin	Prefecto	Edificio, lista ordenada de puntos, avance y acciones.
Revision de Punto	Prefecto	Marcar revisado, resultado, nota y opcion de incidencia.
Nueva Incidencia	Prefecto	Tipo, prioridad, descripcion, punto y evidencia (opcional).
Historial	Prefecto / Admin	Filtros por fecha, estado y edificio.
Dashboard Admin	Admin / Coordinacion	KPIs, rondines por estado, incidencias abiertas y ultimos movimientos.
Gestion de Catalogos	Admin	Usuarios, edificios, puntos, tipos de incidencia y asignaciones.

12. MVP recomendado para una demostracion en clase

La beta no tiene que resolver todos los requisitos futuros. Debe demostrar el flujo principal de forma estable y entendible.

- Login con roles.
- Datos semilla: 2-3 edificios, puntos de revision y 2 prefectos ficticios.
- Creacion/asignacion de un rondin.
- Inicio y cierre del rondin.
- Marcado de puntos como revisados con timestamp.
- Registro de una incidencia vinculada al punto.
- Cambio de estado de la incidencia desde el panel administrativo.
- Historial y dashboard basico.

> **Demostracion recomendada**
> En clase, mostrar un prefecto iniciando un rondin, revisando 2-3 puntos, generando una incidencia de ejemplo y, despues, entrar como coordinacion para ver el rondin e iniciar el seguimiento de esa incidencia.

13. Fuera del MVP / para fases posteriores

- GPS continuo o monitoreo de ubicacion en segundo plano.
- Reconocimiento facial o biometria.
- Integracion con el checador de la UANL.
- Integracion con credenciales institucionales/SSO hasta conocer la infraestructura real.

- Notificaciones push avanzadas.
- Analitica avanzada y predicciones.
- Aplicacion movil nativa separada si la web responsiva resulta suficiente.
- QR/NFC como mecanismo obligatorio de validacion hasta que la ingeniera defina el procedimiento.

14. Preguntas criticas para la ingeniera

Estas son las preguntas que mas impacto tendran en la arquitectura y que conviene llevar a la siguiente reunion:

8. Cuantos prefectos existen y cuantos pueden realizar rondines simultaneamente?
9. Como se asigna un rondin: por edificio, turno, horario fijo o decision del supervisor?
10. Cuantos rondines debe realizar cada prefecto y con que frecuencia?
11. Que puntos exactos deben revisar en cada edificio?
12. El recorrido debe seguir un orden obligatorio?
13. Como se comprobara que el prefecto estuvo fisicamente en cada punto: manual, QR, NFC, GPS u otro mecanismo?
14. Que informacion minima debe contener un reporte de incidencia?
15. Se necesita fotografia/evidencia en todos los reportes o solo en algunos?
16. Quien recibe y atiende las incidencias? Existe un responsable por tipo de problema?
17. Que estados necesita una incidencia y quien puede cambiarlos?
18. Cuanto tiempo debe conservarse el historial?
19. Quienes tendran acceso a la informacion y que filtros necesitan?
20. Debe existir una app movil nativa o basta una aplicacion web adaptable a celular?
21. La autenticacion sera propia del sistema o se utilizara una cuenta institucional UANL?
22. Donde se alojara la aplicacion y la base de datos?

15. Puntos criticos y riesgos

Riesgo	Impacto	Mitigacion inicial
Requisitos incompletos	Alto	Separar requisitos confirmados de supuestos; usar MVP configurable.
Validacion fisica del recorrido no definida	Alto	No acoplar la beta a un metodo unico; preparar Checkpoint/VerificationMethod.
Cambios de operacion despues de programar	Medio/alto	Catalogos y asignaciones configurables en BD.
Permisos mal implementados	Alto	Autorizacion en servidor y pruebas por rol.
Perdida de evidencia/historial	Alto	Auditoria, timestamps, restricciones de edicion de registros cerrados.
Aplicacion dificil de usar en telefono	Medio	UI responsiva y acciones principales grandes/simples.
Fotos/archivos elevan complejidad	Medio	Hacer almacenamiento desacoplado y dejar la evidencia como opcional inicialmente.

16. Brief / prompt para Codex

Instruccion general para Codex

Usa el presente documento como contexto inicial, no como especificacion final. No inventes requisitos institucionales. Cuando una decision este marcada como PENDIENTE, crea una implementacion flexible o deja un punto de extension y documenta la suposicion. Prioriza un MVP ejecutable, limpio y facil de modificar.

CONTEXTO DEL PROYECTO

Desarrollaremos un sistema para FIME-UANL enfocado exclusivamente en la gestion de rondines de prefectos y en el registro/seguimiento de incidencias detectadas durante esos recorridos. El equipo eligio C#/.NET. La ingeniera responsable aun no ha confirmado todos los requisitos operativos.

OBJETIVO DE ESTA ITERACION

Crear una base funcional y ejecutable del sistema, con arquitectura preparada para cambios. Debe permitir autenticar usuarios por rol, administrar edificios/puntos, asignar un rondin, ejecutarlo, registrar revisiones de puntos, crear incidencias y consultarlas desde un panel administrativo.

STACK DE TRABAJO PROPUESTO

- .NET 10 LTS
- ASP.NET Core
- Blazor Web App responsiva (eleccion provisional)
- Entity Framework Core 10
- SQL Server (provisional)
- ASP.NET Core Identity para autenticacion/roles

ROLES MVP

- Prefecto
- Coordinacion/Supervisor
- Administrador

ENTIDADES BASE

User, Role, Building, Checkpoint, Rondin, CheckpointCheck, Incident, IncidentType, IncidentHistory, RondinAssignment.

FLUJO PRINCIPAL

Login -> ver rondin asignado -> iniciar rondin -> revisar puntos -> crear incidencia si corresponde -> continuar -> cerrar rondin -> revisar historial/panel administrativo.

REGLAS CLAVE

- Guardar timestamps de inicio, revisiones y cierre.
- Una incidencia debe vincularse al rondin y, cuando aplique, al punto de revision.
- Estados iniciales de incidencia: Pendiente, En proceso, Resuelta.
- Los edificios y puntos son datos configurables, no hardcodeados.
- La autorizacion debe comprobarse en servidor.
- Los registros cerrados no deben editarse libremente por un prefecto.
- Preparar el modelo para un futuro mecanismo de verificacion fisica (por ejemplo QR), pero NO asumir que QR ya es requisito.
- No implementar GPS continuo, biometria ni integracion con el checador en esta fase.

ENTREGABLE TECNICO DE ESTA ITERACION

1. Crear la solucion .sln y proyectos por capas.
2. Configurar persistencia y migraciones de EF Core.
3. Configurar Identity y roles.
4. Crear entidades, DbContext, configuracion y datos seed de prueba.
5. Crear navegacion y pantallas MVP.
6. Implementar flujo completo de rondin.

7. Implementar incidencias y cambio de estado.
8. Crear dashboard administrativo basico.
9. Agregar validaciones, manejo de errores y pruebas esenciales.
10. Crear README con instrucciones para ejecutar localmente y una seccion llamada DECISIONES PENDIENTES donde se enumeren las decisiones que dependen de la ingeniera.

IMPORTANTE
Antes de introducir librerias adicionales, justificar por que son necesarias. Mantener el proyecto simple. Usar nombres claros y consistentes en español o en ingles, pero elegir un solo idioma para el codigo y mantenerlo de forma consistente. No crear funcionalidades de vigilancia de otros empleados que no forman parte del alcance actual.

17. Ruta sugerida de trabajo con Codex

23. Paso 1: crear la solucion y conseguir que compile/viva en local.
24. Paso 2: autenticar roles y crear seed data.
25. Paso 3: implementar edificios y checkpoints.
26. Paso 4: implementar asignaciones y ejecucion del rondin.
27. Paso 5: implementar incidencias y su historial.
28. Paso 6: dashboard administrativo y filtros.
29. Paso 7: pruebas, validaciones y limpieza del codigo.
30. Paso 8: revisar con la ingeniera los requisitos pendientes y ajustar sin rehacer la base.

18. Criterios de aceptacion para una beta

- Un usuario puede iniciar sesion y solo ve las funciones de su rol.
- Un prefecto puede identificar su rondin y abrirlo.
- El sistema registra inicio, avance y cierre del rondin.
- Cada punto revisado conserva fecha/hora y resultado.
- El prefecto puede registrar al menos una incidencia durante el recorrido.
- La incidencia queda vinculada al rondin y al punto cuando corresponda.
- Coordinacion puede consultar el rondin y la incidencia desde otro usuario.
- Coordinacion puede cambiar el estado de una incidencia y queda historial del cambio.
- Los datos de prueba se pueden cargar sin depender de servicios externos.
- El proyecto puede ejecutarse localmente siguiendo el README.

19. Referencias tecnicas verificadas

Microsoft Learn - ASP.NET Core overview: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0

Microsoft Learn - .NET 10: What's new / LTS: https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview

Microsoft Learn - EF Core overview: https://learn.microsoft.com/en-us/ef/core/

Microsoft Learn - EF Core 10: https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-10.0/whatsnew

Microsoft Learn - ASP.NET Core Identity: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-10.0

Documento de arranque - sujeto a validacion de requisitos